LINUX Commands

1. You have a file with permissions -rw-r--r--, and you run chmod +x file.sh. What happens?
- Original: -rw-r--r--
- After chmod +x file.sh: -rwxr-xr-x
- File becomes executable by all owner,group and others.

Original permissions:
-rw-r--r—

 Owner: rw- → read & write

 Group: r-- → read-only

 Others: r-- → read-only

Command:
chmod +x file.sh

This means:

- **Add execute (x) permission** to **owner, group, and others**.
- It does **not remove** existing permissions—just adds x.

Resulting permissions:

-rwxr-xr-x

 Owner: rwx → read, write, execute

 Group: r-x → read & execute

 Others: r-x → read & execute

```
localhost:~# ls -l
total 16
-rw-r--r--    1 root     root           114 Jul  5  2020 bench.py
-rw-r--r--    1 root     root            76 Jul  3  2020 hello.c
-rw-r--r--    1 root     root            22 Jun 26  2020 hello.js
-rw-r--r--    1 root     root           151 Jul  5  2020 readme.txt
localhost:~# ls -a
.             .ash_history  .mozilla      bench.py      hello.js
..            .cache        .wine         hello.c       readme.txt
localhost:~# ls -r
readme.txt  hello.js    hello.c     bench.py
localhost:~# ls -t
readme.txt  bench.py    hello.c     hello.js
localhost:~# ls -lart
total 40
-rw-r--r--    1 root     root            22 Jun 26  2020 hello.js
-rw-r--r--    1 root     root            76 Jul  3  2020 hello.c
-rw-r--r--    1 root     root           114 Jul  5  2020 bench.py
-rw-r--r--    1 root     root           151 Jul  5  2020 readme.txt
drwx------    3 root     root            61 Jul  6  2020 .cache
drwx------    5 root     root           124 Jul  6  2020 .mozilla
drwxr-xr-x    4 root     root           202 Jul  6  2020 .wine
drwxr-xr-x    5 root     root           237 Jan  9  2021 .
drwxrwxrwx   21 root     root           461 Jun 28 12:03 ..
-rw-------    1 root     root            33 Jun 28 12:03 .ash_history
localhost:~# chmod +x hello.js
```

```
localhost:~# chmod +x hello.js
localhost:~# ls -lart
total 40
-rwxr-xr-x    1 root     root            22 Jun 26  2020 hello.js
-rw-r--r--    1 root     root            76 Jul  3  2020 hello.c
-rw-r--r--    1 root     root           114 Jul  5  2020 bench.py
-rw-r--r--    1 root     root           151 Jul  5  2020 readme.txt
drwx------    3 root     root            61 Jul  6  2020 .cache
drwx------    5 root     root           124 Jul  6  2020 .mozilla
drwxr-xr-x    4 root     root           202 Jul  6  2020 .wine
drwxr-xr-x    5 root     root           237 Jan  9  2021 .
drwxrwxrwx   21 root     root           461 Jun 28 12:04 ..
-rw-------    1 root     root            60 Jun 28 12:04 .ash_history
localhost:~#
localhost:~#
```

2. What is the difference between chmod 744 file.txt and chmod u=rwx,go=r file.txt?

- Both result in: -rwxr--r--

- chmod 744 is octal; chmod u=rwx,go=r is more like assignment.

Feature	chmod 744 file.txt	chmod u=rwx,go=r file.txt
Permission result	-rwxr--r--	-rwxr--r--
Method	**Octal (numeric)**	**Symbolic (textual)**
Meaning	Sets:	Sets:
	- User: **7 = rwx**	- u=rwx: user gets read, write, execute
	- Group: **4 = r--**	- g=r: group gets read only
	- Others: **4 = r--**	- o=r: others get read only
Style	Concise	Verbose, human-readable
Behavior	**Overwrites all permission bits**	**Also overwrites**, but can be more specific
Use case	Ideal for scripting and quick setting	Useful for clarity and readability

Octal Breakdown: chmod 744

- Each digit = sum of permission bits (r=4, w=2, x=1)
- 744 →
  - User: 7 = 4+2+1 → **rwx**
  - Group: 4 → **r--**
  - Others: 4 → **r--**
- Command:

chmod 744 file.txt

Symbolic Breakdown: chmod u=rwx,go=r

- This assigns:
  - u=rwx → user gets read, write, execute
  - g=r → group gets only read
  - o=r → others get only read
- Command:

chmod u=rwx,go=r file.txt

Both commands result in the **same permissions**: -rwxr--r--, but:

- 744 is **short and script-friendly**

- u=rwx,go=r is **clear and self-descriptive**

```
localhost:/# ls -lt
total 52
-rw-r--r--    1 root     root             0 Jun 28 12:42 example2.py
-rw-r--r--    1 root     root             0 Jun 28 12:41 example.py
drwxr-xr-x    4 root     root           140 Jun 28 12:02 run
drwxr-xr-x    4 root     root          2240 Jun 28 12:02 dev
dr-xr-xr-x   12 root     root             0 Jun 28 12:02 sys
dr-xr-xr-x   40 root     root             0 Jun 28 12:02 proc
drwxr-xr-x   39 root     root          1979 Jan  9  2021 etc
drwxr-xr-t    5 root     root           237 Jan  9  2021 root
drwxr-xr-x    3 root     root            58 Jan  9  2021 opt
drwxrwxrwt    2 root     root            37 Jan  9  2021 tmp
drwxr-xr-x    2 root     root          2089 Jan  9  2021 bin
drwxr-xr-x    2 root     root          2312 Nov 21  2020 sbin
drwxr-xr-x   16 root     root           348 Aug 27  2020 var
drwxr-xr-x    8 root     root           901 Aug 18  2020 lib
drwxr-xr-x    2 root     root            37 Jul  5  2020 home
drwxr-xr-x   10 root     root           229 Jun 24  2020 usr
drwxr-xr-x    5 root     root           102 May 29  2020 media
drwxr-xr-x    2 root     root            37 May 29  2020 mnt
drwxr-xr-x    2 root     root            37 May 29  2020 srv
localhost:/# chmod 744 example.py
localhost:/# chmod u=rwx,go=r example2.py
localhost:/# ls -lt
total 52
-rwxr--r--    1 root     root             0 Jun 28 12:42 example2.py
-rwxr--r--    1 root     root             0 Jun 28 12:41 example.py
drwxr-xr-x    4 root     root           140 Jun 28 12:02 run
drwxr-xr-x    4 root     root          2240 Jun 28 12:02 dev
dr-xr-xr-x   12 root     root             0 Jun 28 12:02 sys
```

3. What is the sticky bit, and when should you use it?
- It is directory level.
- Sticky bit (t) on directories allows only the file owner to delete their files.
- Used in shared directories like /tmp.
- Command: chmod +t /dir
- when to use : When there are many users, enable sticky bit so that they don't accidentally delete each others files.

What is the Sticky Bit?

- The **sticky bit** is a special **permission applied to directories**.

- When set, it allows **only the file owner (or root)** to delete or rename files **within that directory**, even if other users have write access to the directory.

Why is the Sticky Bit Important?

- Without the sticky bit, **any user with write access** to a shared directory can **delete or rename others' files**.
- With the sticky bit, **only the file owner or root** can delete/rename their own files.

Typical Use Case

- **/tmp** is a classic example:

ls -ld /tmp

Output:

drwxrwxrwt 10 root root 4096 Jun 26 11:00 /tmp
↑
sticky bit (t)

Many users write to /tmp, but only the file owners can delete their own files.

How to Set the Sticky Bit

sudo chmod +t /shared/dir

How to Verify Sticky Bit

ls -ld /shared/dir

You'll see a t in the last position of the **directory's execute bits**:

drwxrwxrwt 2 root root 4096 Jun 26 11:30 /shared/dir

Without Sticky Bit

drwxrwxrwx 2 root root 4096 Jun 26 11:30 /shared/dir

- Any user can delete anyone else's files.

Feature	With Sticky Bit (+t)	Without Sticky Bit
Who can delete	Only **owner** or **root**	**Anyone** with write access
Use case	Shared folders like /tmp, /var/tmp	Temporary or user-shared directories
Set command	chmod +t /dir	chmod -t /dir to remove it

```
localhost:/# mkdir stickydirrectory
localhost:/# ls -l
total 56
drwxr-xr-x    2 root     root          2089 Jan  9  2021 bin
drwxr-xr-x    4 root     root          2240 Jun 28 12:02 dev
drwxr-xr-x   39 root     root          1979 Jan  9  2021 etc
-rwxr--r--    1 root     root             0 Jun 28 12:41 example.py
-rwxr--r--    1 root     root             0 Jun 28 12:42 example2.py
drwxr-xr-x    2 root     root            37 Jul  5  2020 home
drwxr-xr-x    8 root     root           901 Aug 18  2020 lib
drwxr-xr-x    5 root     root           102 May 29  2020 media
drwxr-xr-x    2 root     root            37 May 29  2020 mnt
drwxr-xr-x    3 root     root            58 Jan  9  2021 opt
dr-xr-xr-x   40 root     root             0 Jun 28 12:02 proc
drwxr-xr-t    5 root     root           237 Jan  9  2021 root
drwxr-xr-x    4 root     root           140 Jun 28 12:02 run
drwxr-xr-x    2 root     root          2312 Nov 21  2020 sbin
drwxr-xr-x    2 root     root            37 May 29  2020 srv
drwxr-xr-x    2 root     root            37 Jun 28 13:05 stickydirrectory
dr-xr-xr-x   12 root     root             0 Jun 28 12:02 sys
drwxrwxrwt    2 root     root            37 Jan  9  2021 tmp
drwxr-xr-x   10 root     root           229 Jun 24  2020 usr
drwxr-xr-x   16 root     root           348 Aug 27  2020 var
```

```
localhost:/# chmod +t stickydirrectory
localhost:/# ls -lt
total 56
drwxr-xr-t    2 root     root            37 Jun 28 13:05 stickydirrectory
-rwxr--r--    1 root     root             0 Jun 28 12:42 example2.py
-rwxr--r--    1 root     root             0 Jun 28 12:41 example.py
drwxr-xr-x    4 root     root           140 Jun 28 12:02 run
drwxr-xr-x    4 root     root          2240 Jun 28 12:02 dev
dr-xr-xr-x   12 root     root             0 Jun 28 12:02 sys
dr-xr-xr-x   40 root     root             0 Jun 28 12:02 proc
drwxr-xr-x   39 root     root          1979 Jan  9  2021 etc
drwxr-xr-t    5 root     root           237 Jan  9  2021 root
drwxr-xr-x    3 root     root            58 Jan  9  2021 opt
drwxrwxrwt    2 root     root            37 Jan  9  2021 tmp
drwxr-xr-x    2 root     root          2089 Jan  9  2021 bin
drwxr-xr-x    2 root     root          2312 Nov 21  2020 sbin
drwxr-xr-x   16 root     root           348 Aug 27  2020 var
drwxr-xr-x    8 root     root           901 Aug 18  2020 lib
drwxr-xr-x    2 root     root            37 Jul  5  2020 home
drwxr-xr-x   10 root     root           229 Jun 24  2020 usr
drwxr-xr-x    5 root     root           102 May 29  2020 media
drwxr-xr-x    2 root     root            37 May 29  2020 mnt
drwxr-xr-x    2 root     root            37 May 29  2020 srv
```

4. You are told to give the owner full access, group only execute, and others no permissions. What symbolic command achieves this?
- chmod u=rwx,g=x,o= file.txt → -rwx--x---

- **Owner** → full access (read, write, execute)
- **Group** → execute only
- **Others** → no permissions

Symbolic Command:
chmod u=rwx,g=x,o= file.txt

User Type	Permission Description	Symbolic	Binary	Result
u	User/Owner: Read, Write, Execute	rwx	111	Full access
g	Group: Execute only	x	001	Run-only
o	Others: No permission	(none)	000	Denied

Resulting File Mode:
-rwx--x---

Explanation:

- rwx → owner can **read/write/execute**
- --x → group can **only execute**
- --- → others have **no access**

Alternate Numeric (Octal) Form:
chmod 710 file.txt

Section	Binary	Octal	Description
User	111	7	Read, write, execute
Group	001	1	Execute only
Others	000	0	No permissions

```
localhost:/q4# touch readme.txt
localhost:/q4# ls -l
total 0
-rw-r--r--    1 root     root             0 Jun 28 13:14 readme.txt
-rwx--x---    1 root     root             0 Jun 28 13:13 testfile
localhost:/q4# chmod u=rwx,g=x,o= readme.txt
localhost:/q4# ls -l readme.txt
-rwx--x---    1 root     root             0 Jun 28 13:14 readme.txt
localhost:/q4# ls -l
total 0
-rwx--x---    1 root     root             0 Jun 28 13:14 readme.txt
-rwx--x---    1 root     root             0 Jun 28 13:13 testfile
localhost:/q4#
```

5. What is umask, and why is it important?
- umask is the default permission mask for new files and directories.
- Subtracted from default: 666 (files), 777 (dirs)
- Ensures secure default permissions.

What is umask?

- umask stands for **"user file-creation mode mask"**.
- It controls the **default permission bits** for **new files and directories** created by the user.
- Instead of setting permissions, umask **masks (subtracts)** permissions from the system default.

Default Permissions Before umask

Type	Default Permission	In Binary	Octal
File	rw-rw-rw-	666	Read & write for all

Type	Default Permission	In Binary	Octal
Directory	rwxrwxrwx	777	Full access for all

Why 666 for files? Because **new files are not executable by default**.

What umask Does

- It **subtracts** permissions from the default using a bitwise AND NOT operation.
- For example:
  - Default file: 666
  - umask: 022
  - Final permission: 644 → rw-r--r--

Why Is umask Important?

- Ensures **secure defaults**:
  - Prevents newly created files from being world-writable or group-writable unintentionally.
- Used in:
  - Shell sessions
  - Scripts
  - Multi-user environments

Common umask Values

umask	Files → Result	Dirs → Result	Used For
022	644 → rw-r--r--	755 → rwxr-xr-x	Typical default (safe sharing)
002	664 → rw-rw-r--	775 → rwxrwxr-x	Collaborative dev environment
027	640 → rw-r-----	750 → rwxr-x---	More private setting
077	600 → rw-------	700 → rwx------	Maximum privacy/security

Summary

- umask controls the **default permissions** for new files and directories.
- It **subtracts** permissions from system defaults (666 for files, 777 for dirs).

- Helps ensure **secure, predictable** file creation, especially in multi-user environments.

```
localhost:/# mkdir q5
localhost:/# cd q5
localhost:/q5# touch file.txt
localhost:/q5# mkdir newdir
localhost:/q5# ls -l
total 4
-rw-r--r--    1 root     root             0 Jun 28 13:21 file.txt
drwxr-xr-x    2 root     root            37 Jun 28 13:22 newdir
localhost:/q5# ls -l file.txt
-rw-r--r--    1 root     root             0 Jun 28 13:21 file.txt
localhost:/q5# ls -ld newdir
drwxr-xr-x    2 root     root            37 Jun 28 13:22 newdir
localhost:/q5#
```

6. If the umask is 022, what are the default permissions for a new file and a new directory?
- File: 666 - 022 = 644 → -rw-r--r--
- Dir: 777 - 022 = 755 → drwxr-xr-x

Default Permissions

Item	Default Permissions	Octal
File	rw-rw-rw-	666
Directory	rwxrwxrwx	777

umask: 022

- In octal:
  0 = no mask
  2 = mask write (w)
  2 = mask write (w)

New File:

666 (rw-rw-rw-)
− 022 (---w--w-)
= 644 (rw-r--r--)

Final file permissions:

-rw-r--r—

New Directory:

777 (rwxrwxrwx)
− 022 (---w--w-)
= 755 (rwxr-xr-x)

Final directory permissions:

drwxr-xr-x

Type	Default	umask	Result	Symbolic
File	666	022	644	-rw-r--r--
Directory	777	022	755	drwxr-xr-x

Examples with Different umask Values

umask 002 — Collaborative Environments

Type	Default	umask	Result	Symbolic
File	666	002	664	-rw-rw-r--
Dir	777	002	775	drwxrwxr-x

Used in: Development teams where users are in the same group and should share files.

umask 027 — Restricted Group Access

Type	Default	umask	Result	Symbolic
File	666	027	640	-rw-r-----
Dir	777	027	750	drwxr-x---

Used in: Production environments where group access is limited and others are denied access.

umask 077 — Private/Secure

Type	Default	umask	Result	Symbolic
File	666	077	600	-rw-------
Dir	777	077	700	drwx------

Used in: Secure systems, root accounts, or sensitive data where files must be private to the owner.

Conclusion

umask	File Perms	Dir Perms	Description
002	-rw-rw-r--	drwxrwxr-x	Group collaboration
022	-rw-r--r--	drwxr-xr-x	Standard/default secure setup
027	-rw-r-----	drwxr-x---	Group-restricted, secure
077	-rw-------	drwx------	Private for owner only

7. Why is umask often set to 002 in development environments but 027 or 077 in production?
- 002: Collaboration (Dev) → Files: 664, Dirs: 775
- 027: Limited (Prod) → Files: 640, Dirs: 750
- 077: Private (Secure) → Files: 600, Dirs: 700

umask controls default file and directory permissions when new files/directories are created.

umask 002 – **Development (Collaborative)**

Result **File: 664 (-rw-rw-r--) Dir: 775 (drwxrwxr-x)**

✓ Owner Read, write

✓ Group Read, write (collaboration within the same group)

✗ Others Read-only

Why in Dev?

- Multiple developers often belong to the **same group**.
- Allows group members to **edit and share** files easily.
- Suitable when shared access is required **within a team**.

Example:

```
localhost:/# mkdir q7
localhost:/# cd q7
localhost:/q7# umask 002
localhost:/q7# touch test.txt
localhost:/q7# mkdir newdir
localhost:/q7# ls -l
total 4
drwxrwxr-x    2 root     root            37 Jun 28 13:33 newdir
-rw-rw-r--    1 root     root             0 Jun 28 13:33 test.txt
localhost:/q7#
```

umask 077 – **Highly Secure/Private**

Result **File: 600 (-rw-------) Dir: 700 (drwx------)**

✓ Owner Full access

✗ Group No access

✗ Others No access

Why in Secure Systems?

- Used in environments handling **sensitive data** (e.g., security keys, backups, config).
- Ensures files are **completely private** to the owner.

- Often used by **root or system-level processes**.

Environment	umask	File Perms	Dir Perms	Use Case / Rationale
Development	002	664	775	Team collaboration, shared editing
Production	027	640	750	Secure with limited group visibility
High Security	077	600	700	Private owner-only access

```
localhost:/q7# umask 077
localhost:/q7# touch test2.txt
localhost:/q7# mkdir newdir1
localhost:/q7# ls -l
total 8
drwxrwxr-x    2 root     root            37 Jun 28 13:33 newdir
drwx------    2 root     root            37 Jun 28 13:36 newdir1
-rw-rw-r--    1 root     root             0 Jun 28 13:33 test.txt
-rw-------    1 root     root             0 Jun 28 13:36 test2.txt
localhost:/q7#
```

8. useradd vs adduser

Feature	useradd	adduser
Type	Low-level system command (binary)	High-level interactive script (Perl or Python)
Availability	Universal across Linux distros	Common on **Debian/Ubuntu** (may not exist on RHEL/CentOS)
Interactivity	Non-interactive	Interactive prompts
Default behavior	Minimal (requires options)	Automatically sets home, shell, group, etc.
Home Dir	❌ Not created by default (-m needed)	✅ Created by default
Password	❌ Must be set separately	✅ Prompts during user creation

useradd — Low-Level Command

Description:

- Adds a user by directly editing system files like /etc/passwd, /etc/shadow, /etc/group.
- No password or home directory unless specified.

Syntax:

sudo useradd [options] username

Common Options:

Option	Description
-m	Create a home directory
-d	Specify custom home directory
-s	Set login shell
-G	Add user to additional groups
-u	Set user ID
-e	Set account expiration date

Example:

sudo useradd -m -s /bin/bash -G developers john
sudo passwd john # Set password manually

adduser — High-Level Script

Description:

- Wrapper around useradd with interactive prompts.
- Sets home directory, shell, user group, and password automatically.
- Easier and more user-friendly.

Syntax:

sudo adduser username

Example:

sudo adduser alice

Prompts for:

- Full name
- Room number
- Phone numbers
- Password and confirmation

Creates:

- /home/alice
- Adds entry to /etc/passwd
- Sets default shell to /bin/bash
- Adds user to its own group (alice)

Comparison Summary

Feature	useradd	adduser
User-friendly?	❌ No	✅ Yes
Home dir created?	❌ No (need -m)	✅ Yes
Password setup?	❌ No (passwd needed)	✅ Yes (prompted)
Group setup?	❌ Must use -G	✅ Done automatically
Script vs Binary	Binary	Script (Perl or Python)
Suitable for scripting	✅ Ideal	❌ Interactive

When to Use What?

Use Case	Recommended Command
Manual user creation (quick)	adduser
Automated scripts or configs	useradd
Minimalist system (RHEL, Arch)	useradd

Use Case	Recommended Command
Ease and prompts (Debian/Ubuntu)	adduser